TRANSURBAN — IA: apresentação do CSV em 5 minutos

Jullyany e Camila • Ciência de Dados e API • Sem slides

Duração aproximada: 5 minutos, contando as consultas. Ensaiem uma vez com cronômetro. Leiam somente os parágrafos FALA; NA TELA é orientação para quem opera. José Henrique pode fazer os cliques. As duas apresentam somente IA, sem demonstrar cadastros de DS ou o site.

Antes de começar: API funcionando, terminal aberto e navegador do mesmo computador em http://127.0.0.1:8000/docs. Deixe GET /ia/dados-publicos/fonte já executado. O CSV usado é programacao_curitiba.csv, incluído no pacote.

0:00–0:45 — Jullyany: abertura e fonte

NA TELA: mostre a resposta de GET /ia/dados-publicos/fonte. Aponte cidade e período, sem ler o JSON inteiro.

FALA: “Bom dia, pessoal! Eu sou a Jullyany e, junto com a Camila, vou apresentar a análise do CSV do TRANSURBAN. Nosso objetivo foi entender os dados, verificar sua qualidade e disponibilizar indicadores pela API.

FALA: Usamos uma base pública histórica de programação do transporte de Curitiba como estudo externo para o projeto. Ela foi publicada em um GTFS comunitário e não é uma publicação oficial da URBS. O período declarado vai de 28 de setembro a 5 de outubro de 2026.”

0:45–1:35 — Jullyany: o que o CSV representa

FALA: “O CSV é um arquivo organizado como uma tabela: as colunas representam informações, e cada linha representa um registro.

FALA: Nosso arquivo tem nove colunas, incluindo identificador da programação, código e nome da linha, tipo de serviço, operadora, cidade e período da fonte.

FALA: Uma linha do arquivo representa um registro de programação publicado. Não representa um passageiro nem comprova uma viagem realizada.

FALA: O CSV reúne informações relacionadas no mesmo registro, como o nome da linha e a operadora. Usamos Python e Pandas para ler essa tabela, preservar os códigos como texto e organizar a análise.”

1:35–2:15 — Jullyany: verificação da qualidade

NA TELA: execute GET /ia/dados-publicos/qualidade. Mostre ausentes_por_coluna, ids_duplicados e apta_para_calculos_do_snapshot.

FALA: “Antes dos cálculos, verificamos as colunas, os campos vazios, os identificadores repetidos e a consistência entre códigos e nomes.

FALA: Também conferimos o hash dos arquivos, que ajuda a identificar alterações no conteúdo. Essa assinatura não prova que as viagens aconteceram; ela verifica a

integridade do arquivo.

FALA: Se houver uma divergência nessas verificações, a API bloqueia os indicadores para revisão. Agora a Camila vai mostrar os resultados.”

2:15–3:15 — Camila: perguntas e indicadores

NA TELA: execute GET /ia/dados-publicos/resumo. Aponte um campo por vez.

FALA: “Nossas perguntas foram: quantos registros existem, quantas linhas aparecem e como a programação está distribuída entre elas?

FALA: O arquivo possui 71.086 registros de programação e 309 códigos de linha distintos. Ao somar as contagens dos grupos de linha, voltamos ao total de registros.

FALA: A média é aproximadamente 230,05 registros por linha: dividimos o total de registros pela quantidade de linhas.

FALA: A mediana é 162. Para encontrá-la, ordenamos as quantidades e identificamos o valor central. A média pode ser influenciada por linhas com quantidades maiores, enquanto a mediana ajuda a observar o centro da distribuição.”

3:15–4:05 — Camila: agrupamento e comparação

NA TELA: abra GET /ia/dados-publicos/agrupamentos/{dimensao}. Preencha dimensao = linha e limite = 5. Clique Try it out e Execute, conforme a interface permitir. O nome do parâmetro é limite.

FALA: “Aqui agrupamos os registros por linha. Agrupar significa reunir registros com uma característica em comum para comparar suas quantidades.

FALA: Mostramos cinco grupos para facilitar a leitura no telão. A API continua identificando o total completo e a quantidade exibida.

FALA: Também é possível agrupar por serviço: dias úteis, sábado e domingo. Os indicadores descrevem o conjunto do arquivo; não são médias diárias nem viagens comprovadamente realizadas.”

4:05–4:45 — Camila: JSON e limitações

NA TELA: volte ao resumo e mostre atraso_medio_min e viagens_realizadas, ambos null.

FALA: “A API entrega os resultados em JSON, um formato que um dashboard pode utilizar para construir gráficos e tabelas.

FALA: O atraso médio aparece como null porque essa informação não está disponível na fonte. Isso não significa atraso zero.

FALA: Portanto, nossa análise não mede atrasos reais de Maringá ou Sarandi e não prevê quando um ônibus vai chegar. Ela descreve a programação publicada de Curitiba.”

4:45–5:00 — Jullyany: encerramento

FALA: “Com esse trabalho, aprendemos a entender o CSV, verificar sua qualidade, produzir indicadores e explicar seus limites. Esses resultados podem orientar a próxima etapa do dashboard. Obrigada pela atenção!”

Cola de consulta — não faz parte da fala

Consulta	O que mostrar
/ia/dados-publicos/fonte	Curitiba, origem comunitária e período declarado.
/ia/dados-publicos/qualidade	Campos vazios, IDs repetidos e integridade.
/ia/dados-publicos/resumo	71.086 registros, 309 códigos, média 230,0518 e mediana 162.
/ia/dados-publicos/agrupamentos/linha?limite=5	Comparação por linha; cinco grupos exibidos.
Resumo novamente	null significa indisponível, não zero.

Se perguntarem se vocês treinaram uma IA: “Não. Nesta avaliação trabalhamos Ciência de Dados: preparação, análise e disponibilização dos indicadores.”

Se perguntarem por que Curitiba: “Não tínhamos uma base local de atrasos reais disponível para essa análise. Usamos uma fonte pública histórica e documentamos a diferença.”

Se a API falhar: identifiquem as evidências anteriores como evidências; não anunciem execução ao vivo que não ocorreu. Os tempos acima são metas de ensaio, não uma garantia de duração.